WaferGuard ML

Architecture & Model Comparison

Wafer Map Defect Classification

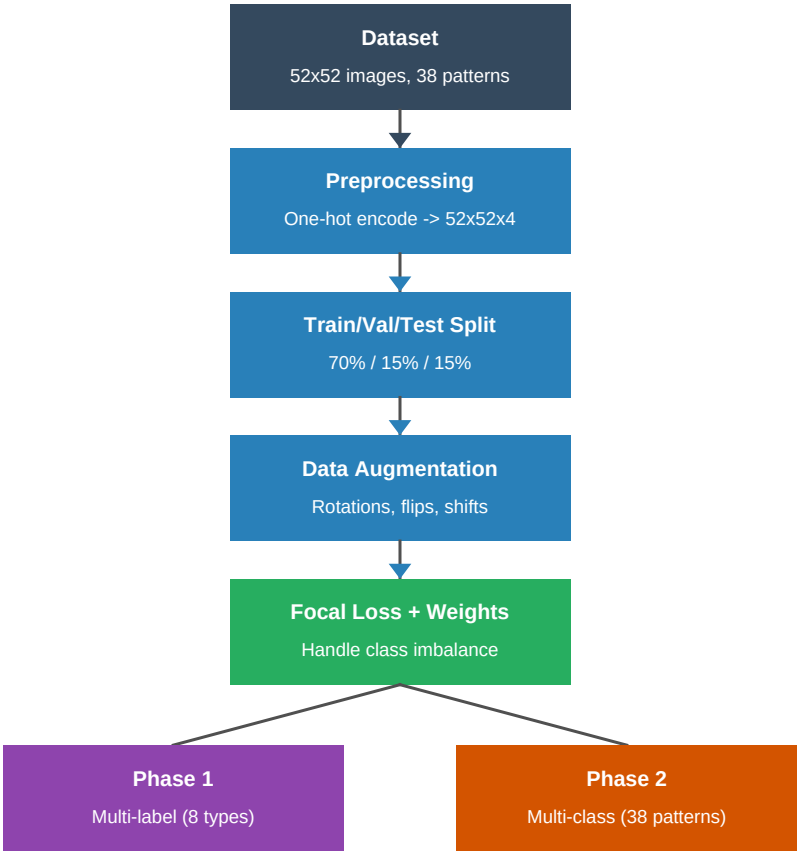
Mixed-Type Defect Dataset

Deep Learning Pipeline: Custom CNN vs. MobileNetV2 Transfer Learning

1. Pipeline Overview

The WaferGuard ML pipeline classifies wafer map defect patterns using a two-phase deep learning approach. The system processes 52x52 pixel wafer map images from the Mixed-Type Wafer Defect Dataset and classifies them into defect patterns.

Data Flow



The pipeline begins with the Mixed-Type Wafer Map Dataset containing 52x52 pixel images. Each pixel represents a discrete state: blank (0), normal die (1), broken die (2), or an undocumented state (3). These are one-hot encoded into 4 channels to preserve their categorical nature -- treating them as intensity values would be semantically incorrect.

The data is split using iterative stratification to ensure all 38 unique defect pattern combinations are represented in train, validation, and test sets. Geometrically valid augmentations (rotations, flips, shifts) are applied only during training, since wafer maps are rotationally symmetric. Elastic deformations and color jitter are avoided because the pixel values are discrete categorical states.

Binary Focal Loss with per-label class weights addresses the severe class imbalance (e.g., ~1000 Normal samples vs. ~200

Near_Full samples). The focal loss down-weights easy/frequent examples so the model focuses on hard/rare defect types.

2. Phase 1 -- Multi-Label Classification

Phase 1 performs multi-label binary classification across the 8 base defect types: Center, Donut, Edge_Loc, Edge_Ring, Loc, Near_Full, Scratch, and Random. Each image can have multiple defect labels simultaneously (e.g., Donut+Scratch). Two architectures are compared.

Model 1A: Custom CNN (424K parameters)

A lightweight CNN designed specifically for the 52x52 input size. The architecture uses 4 convolutional blocks with progressively increasing filter counts:

Layer	Output Shape	Details
Input	52 x 52 x 3	Preprocessed wafer map
Conv Block 1	26 x 26 x 32	Conv2D(32) + BatchNorm + ReLU + MaxPool
Conv Block 2	13 x 13 x 64	Conv2D(64) + BatchNorm + ReLU + MaxPool
Conv Block 3	6 x 6 x 128	Conv2D(128) + BatchNorm + ReLU + MaxPool
Conv Block 4	3 x 3 x 256	Conv2D(256) + BatchNorm + ReLU + MaxPool
GlobalAvgPool	256	Spatial averaging (reduces overfitting vs Flatten)
Dense	128	Fully connected + ReLU
Dropout	128	Rate = 0.5 (regularization)
Output	8	Sigmoid activation (independent per label)

Key design decisions: GlobalAveragePooling2D is used instead of Flatten to reduce the parameter count and mitigate overfitting. BatchNormalization after each convolution stabilizes training. The final layer uses sigmoid (not softmax) because this is multi-label -- each defect type is predicted independently.

Model 1B: MobileNetV2 Transfer Learning (2.6M parameters)

Uses a pre-trained MobileNetV2 backbone (ImageNet weights) as a frozen feature extractor. Since MobileNetV2 expects 224x224x3 RGB inputs, two adapter layers are needed:

Layer	Output Shape	Details
Input	52 x 52 x 4	4-channel one-hot encoded
Conv2D 1x1	52 x 52 x 3	Project 4 channels to 3
Resize	224 x 224 x 3	Bilinear upsampling
MobileNetV2	7 x 7 x 1280	Frozen ImageNet backbone
GlobalAvgPool	1280	Spatial averaging

Dense	128	Fully connected + ReLU
Dropout	128	Rate = 0.5
Output	8	Sigmoid activation

The hypothesis was that ImageNet features (edges, textures, shapes) would transfer to wafer map patterns. However, results show this assumption is weak -- wafer maps use discrete categorical pixels, not natural image intensities, limiting the utility of pre-trained features.

3. Phase 2 -- Multi-Class Classification

Phase 2 extends the classification from 8 base defect types to all 38 unique defect patterns (including single, double, triple, and quadruple defect combinations). It reuses the Phase 1 convolutional backbones as frozen feature extractors and replaces the classification head.

Architecture: Backbone Reuse

Component	Details
Backbone	Phase 1 convolutional layers (frozen)
New Head	Dense(128, ReLU) -> Dropout -> Dense(38, softmax)
Output	38-class probability distribution (mutually exclusive)

Unlike Phase 1's sigmoid outputs (multi-label), Phase 2 uses softmax because each wafer map belongs to exactly one of the 38 defect pattern combinations. The loss function switches from Binary Focal Loss to Sparse Categorical Crossentropy with class weights.

Two-Stage Training Strategy

Stage A -- Head-Only Training

- All convolutional layers from Phase 1 are frozen
- Only the new Dense(128) + Dense(38) head is trained
- Learning rate: 1e-3 (aggressive, since only the head is updating)
- Duration: up to 30 epochs with early stopping
- Purpose: Learn the new 38-class decision boundaries without disrupting learned features

Stage B -- Fine-Tuning

- Last 2 convolutional blocks in the backbone are unfrozen
- End-to-end training with a very low learning rate: 1e-5
- Duration: up to 30 additional epochs

- Purpose: Adapt high-level features to the finer-grained 38-class task

This two-stage approach prevents catastrophic forgetting -- if all layers were unfrozen immediately with a high learning rate, the useful features learned in Phase 1 would be destroyed before the new head learns meaningful gradients.

4. Training Configuration

Loss Functions

Phase 1: Binary Focal Loss

FL(pt) = -alpha_t * (1 - pt)^gamma * log(pt)

Where alpha_t is the per-label weight (higher for rare classes like Near_Full) and gamma=2.0 is the focusing parameter. This down-weights well-classified examples and focuses learning on hard/rare defect types. Combined with inverse-frequency class weights, this addresses the severe imbalance in the dataset.

Phase 2: Sparse Categorical Crossentropy

Standard cross-entropy loss for multi-class classification, combined with per-class weights computed by sklearn's compute_class_weight to handle the 38-class imbalance.

Optimizer & Callbacks

Setting	Value	Purpose
Optimizer	Adam	Adaptive learning rate per parameter
Initial LR	1e-3	Standard starting point
ReduceLROnPlateau	patience=5, factor=0.5	Halve LR when val_loss stalls
EarlyStopping	patience=10, restore_best	Stop when no improvement
Max Epochs (P1)	100	Typically stops early around 30-50
Max Epochs (P2)	30 + 30	Stage A + Stage B

Data Details

Property	Value
Dataset	Mixed-Type Wafer Map Defect Dataset
Image Size	52 x 52 pixels
Pixel Encoding	One-hot: 4 channels (blank, normal, broken, unknown)
Defect Types (base)	8: Center, Donut, Edge_Loc, Edge_Ring, Loc, Near_Full, Scratch, Random
Defect Patterns (total)	38 unique combinations (single through quadruple)

Split Ratio	70% train / 15% validation / 15% test
Stratification	Iterative (preserves all 38 patterns in each split)
Augmentation	Rotations, flips, shifts (training only)

5. Results Comparison

Overall Model Performance

Model	Macro F1	Weighted F1
Phase 1 -- Custom CNN (8-label)	**0.9779**	**0.9905**
Phase 1 -- MobileNetV2 (8-label)	0.8998	0.8847
Phase 2 -- Custom CNN (38-class)	0.9736	0.9735
Phase 2 -- MobileNetV2 (38-class)	0.9667	0.9658

Macro F1 gives equal weight to every class regardless of size, making it the primary metric for imbalanced datasets. Weighted F1 weights each class by its support. The Custom CNN achieves the best scores across both phases and both metrics.

Phase 1 -- Per-Label F1 Scores (8 Base Defect Types)

Defect Type	Custom CNN F1	MobileNetV2 F1	Delta
Center	1.00	0.97	+0.03
Donut	1.00	1.00	0.00
Edge_Loc	0.99	0.75	+0.24
Edge_Ring	0.99	0.88	+0.11
Loc	0.99	0.89	+0.10
Near_Full	0.90	0.88	+0.02
Scratch	0.98	0.84	+0.14
Random	0.97	0.99	-0.02
Macro Avg	0.98	0.90	+0.08

The Custom CNN outperforms MobileNetV2 on 7 of 8 defect types. The largest gaps are on Edge_Loc (+0.24) and Scratch (+0.14), suggesting MobileNetV2's ImageNet features struggle with these particular spatial patterns in discrete-valued wafer maps.

6. Key Takeaways

1. Custom CNN Outperforms Transfer Learning

The lightweight Custom CNN consistently beats MobileNetV2 across both phases. This is likely because wafer map images use discrete categorical pixel values (0/1/2/3), which are fundamentally different from the continuous-intensity natural images that ImageNet features were trained on. The domain gap is too large for transfer learning to help.

2. Efficiency: 6x Fewer Parameters, Better Results

The Custom CNN has only 424K parameters vs. MobileNetV2's 2.6M (a 6x reduction). Despite being much smaller, it achieves higher accuracy. This means faster inference, lower memory usage, and easier deployment -- critical for real-time manufacturing quality control systems.

3. Strong Scaling from 8 Labels to 38 Classes

Phase 2 models maintain near-Phase-1 performance when scaling from 8 base defect types to 38 defect pattern combinations. The Custom CNN backbone achieves 0.97 Macro F1 on the 38-class task, demonstrating that the learned features generalize well to finer-grained classification.

4. Backbone Reuse is Effective

The two-stage training strategy (freeze backbone, train head, then fine-tune) successfully transfers Phase 1 knowledge to the Phase 2 task. This avoids training from scratch and reduces compute requirements.

5. Focal Loss Handles Class Imbalance Well

The combination of Binary Focal Loss with per-label class weights effectively handles the severe class imbalance. Even the rarest class (Near_Full, ~200 samples) achieves 0.90+ F1 scores, showing the model does not simply ignore minority classes.

Generated from wafer_images/architecture.md